

Funzioni

Una funzione è un blocco di istruzioni che viene eseguito solamente quando viene chiamato. Le funzioni sono molto importanti per un buon riutilizzo del codice: si scrivono le istruzioni una volta sola e le si eseguono più volte.

Noi abbiamo già incontrato diverse funzioni: `printf()`, `scanf()`, `strlen()`, lo stesso `main()` è una funzione! Queste sono tutte funzioni predefinite.

Chiaramente possiamo definire delle nostre funzioni, che eseguano le operazioni che ci servono e interessano.

Dichiarazione di una funzione

La dichiarazione della funzione non è altro che il passo in cui andiamo a creare la nostra funzione (indichiamo la sua esistenza).

Sintassi:

```
tipoRestituito nomeFunzione(tipo parametro1, ..., tipo parametroN);
```

Per prima cosa occorre definire il tipo di dato che verrà restituito dalla nostra funzione; i tipi possibili sono quelli che abbiamo già visto per le variabili. Se la nostra funzione non deve restituire niente, indichiamo **void** come tipo di dato.

Il nome della funzione lo scegliamo noi; chiaramente, è opportuno assegnare un nome che indichi a cosa serve la funzione che stiamo dichiarando.

Tra le parentesi tonde, infine, si indicano i parametri della funzione, ovvero tutti i dati che la funzione deve ricevere in ingresso per poter svolgere le operazioni che contiene. I parametri vengono separati da una `,` ed occorre sempre indicare il loro tipo. I parametri, all'interno della funzione, agiscono esattamente come delle variabili.

La dichiarazione della funzione consiste sostanzialmente nel creare il cosiddetto **prototipo** della funzione, indicato sopra il `main()`.

Definizione di una funzione

La definizione è il passaggio in cui andiamo ad indicare il corpo della funzione, ovvero l'insieme delle istruzioni che la compongono.

Sintassi:

```
tipoRestituito nomeFunzione(tipo parametro1, ..., tipo parametroN){  
    // istruzioni  
}
```

Osserviamo come la struttura sia identica a quella della dichiarazione, con l'unica differenza che ora andiamo ad indicare quali sono le istruzioni contenute nella funzione.

La definizione viene fatta sotto al main().

È possibile unire dichiarazione e definizione, indicando direttamente il corpo della funzione sopra al main(), ma per una maggiore comprensione e leggibilità del codice è opportuno separare le due fasi, creando il prototipo della funzione (indicandolo sopra al main()) e definendo il corpo della funzione (sotto al main()).

Chiamare una funzione

Come abbiamo detto, le funzioni che creiamo non vengono eseguite automaticamente, ma devono essere chiamate. Per chiamare una funzione è sufficiente indicare il suo nome e tra le parentesi tonde specificare l'elenco dei parametri necessari alla funzione stessa.

Esempio:

```
// dichiarazione della funzione (prototipo)
tipoRestituito miaFunzione(tipo parametro1, ..., tipo parametroN);

int main(){
    // istruzioni
    miaFunzione(par1, ..., parN); //chiamata della funzione
    // altre istruzioni
    return 0;
}

//definizione della funzione
tipoRestituito miaFunzione(tipo parametro1, ..., tipo parametroN){
    // istruzioni
}
```

Passaggio dei parametri

Abbiamo detto che le funzioni possono avere dei parametri, utilizzati durante l'esecuzione della funzione.

È possibile passare i parametri in due modi diversi: per valore e per riferimento (o referenza).

Attraverso il passaggio di parametri per valore, durante la chiamata a funzione non viene passata l'intera variabile, ma solamente il suo valore.

Questo significa che la funzione non lavorerà direttamente sulla variabile, ma solo sul suo valore; pertanto, qualsiasi modifica che viene fatta a tale valore non si ripercuoterà sul valore della variabile nel main(), ma la modifica è solo locale alla funzione.

Il passaggio per valore viene effettuato indicando semplicemente il nome della variabile quando chiamiamo la funzione.

Tuttavia, a volte può essere comodo che le modifiche effettuate dalla funzione vengano applicate direttamente alla variabile che passiamo come parametro; è qui che entra in gioco il passaggio per riferimento (o referenza), con cui alla funzione viene passato il riferimento (l'indirizzo) della variabile.

Questo significa che tutte le modifiche verranno mantenute anche al di fuori della funzione.

Il passaggio per riferimento si indica in una maniera diversa rispetto al classico passaggio di parametri. Vediamo un esempio:

```
tipoRestituito nomeFunzione(tipo *parametro);

int main(){
// istruzioni
  nomeFunzione(&par); //chiamata della funzione
// altre istruzioni
  return 0;
}
```

Nel prototipo della funzione (e anche nella definizione, non riportata nell'esempio, è necessario aggiungere un * al nome del parametro; questo va ad indicare che il parametro restituito non è un parametro "semplice", ma è un puntatore.

Un puntatore è una variabile particolare, che contiene l'indirizzo di memoria di un'altra variabile.

Quando, all'interno della funzione, dobbiamo utilizzare il valore della variabile puntata dal puntatore, occorre usare l'operatore di deferenza, che è proprio *.

Nella chiamata della funzione, per effettuare il passaggio per riferimento occorre aggiungere & al nome della variabile.